

Programación Multinúcleo y Extensiones SIMD: Implementación y Análisis del Método de Jacobi

ÁLVARO SCHWIEDOP

Arquitectura de Computadores • 2º Grao Enx. Informática
Universidade de Santiago de Compostela
curso1523@usc.es

15 de marzo de 2026

Resumen

Se implementan cuatro versiones del método iterativo de Jacobi en C sobre el supercomputador FinisTerae III del CESGA: versión secuencial de referencia (v1), versión optimizada para caché con división de lazos, desenrollamiento y blocking (v2), versión vectorial con instrucciones AVX256 y FMA (v3), y versión paralela con OpenMP (v4). Para $n=3200$, v3 alcanza un speedup de $1.47\times$ sobre v2, y v4 con 32 hilos consigue $21\times$ sobre la ejecución con 1 hilo. Los experimentos revelan escalabilidad casi lineal hasta 16 hilos y degradación a 32 hilos para tamaños pequeños, atribuible al exceso de sincronización OpenMP.

Palabras clave: Jacobi, AVX256, FMA, OpenMP, cache blocking, speedup, FinisTerae III.

I. INTRODUCCIÓN

El método de Jacobi [1] es un algoritmo iterativo clásico para la resolución de sistemas $Ax=b$ con matriz diagonalmente dominante. Partiendo de $x^{(0)}=0$, en cada iteración se actualiza:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

hasta que $\|x^{(k+1)} - x^{(k)}\|_2 < \text{tol}$. La doble suma implica $\mathcal{O}(n^2)$ operaciones por iteración, lo que hace que tamaños grandes sean costosos y susceptibles de aceleración mediante optimizaciones de bajo nivel.

El objetivo es implementar cuatro versiones de complejidad creciente, medirlas en FinisTerae III (Intel Xeon Platinum 8352Y, 2.2 GHz, 64 núcleos por nodo) y cuantificar el speedup obtenido. La Sección II describe las implementaciones; la Sección III, la metodología experimental; la Sección IV, los resultados; la Sección V, las conclusiones.

II. IMPLEMENTACIONES

A. v1 — Secuencial base

Traducción directa del pseudocódigo del enunciado sin ninguna optimización manual. El algoritmo recibe la matriz A , el vector b , la tolerancia tol y el máximo de iteraciones max_iter ; itera actualizando x hasta convergencia:

Entradas: $A \in \mathbb{R}^{n \times n}$, $b, x \in \mathbb{R}^n$, tol , max_iter
Aux: $x_{\text{new}} \in \mathbb{R}^n$

```
para iter = 0 hasta max_iter:  
    norm2 ← 0  
    para i = 0 hasta n - 1:  
        σ ← 0  
        para j = 0 hasta n - 1:  
            si i ≠ j: σ += a[i][j] · x[j]  
        x_new[i] ← (b[i] - σ) / a[i][i]  
        norm2 += (x_new[i] - x[i])2  
    x ← x_new  
    si √norm2 < tol: terminar  
Salida: imprimir norm2
```

La implementación en C introduce la condición `if (i!=j)` dentro del bucle interno, lo cual genera una rama innecesaria:

```
for (int j = 0; j < n; j++)  
    if (i != j)  
        sigma += a[i*n+j] * x[j];  
x_new[i] = (b[i]-sigma) / a[i*n+i];
```

Se usa como referencia de rendimiento y corrección numérica.

B. v2 — Optimizaciones de caché

Se aplican cuatro mejoras sobre v1:

1. **Reducción de instrucciones:** puntero de fila `const double *row=&a[i*n]`, elimina el recalclo del índice.
2. **División de lazos:** el bucle j se parte en $[0, i)$ e (i, n) , eliminando la rama condicional.
3. **Desenrollamiento $\times 4$:** cuatro acumulaciones explícitas por iteración.
4. **Cache blocking:** bloques de `BLOCK_SIZE=64` filas para mantener x en caché L1/L2.

C. v3 — AVX256 + FMA

Se emplea `__m256d` (4 double/registro) con `_mm256_fmadd_pd`. La memoria se reserva alineada a 32 B con `aligned_alloc` y el *stride* de fila se redondea a múltiplo de 4 (`n_pad`). La reducción horizontal usa `_mm256_hadd_pd`:

```
__m256d va = _mm256_load_pd(&row[j]);
__m256d vx = _mm256_load_pd(&x[j]);
vsigma = _mm256_fmadd_pd(va, vx, vsigma);
/* reduccion horizontal al final */
__m256d h = _mm256_hadd_pd(vsigma, vsigma);
```

No se usan intrínsecos `_u` (no alineados) como exige el enunciado.

D. v4 — OpenMP

La región `parallel` se sitúa *fuera* del bucle de iteraciones evitando la creación repetida de hilos. La reducción de `norm2` usa variables locales por hilo más `#pragma omp atomic`; la copia $x \leftarrow x_{\text{new}}$ y el test de convergencia, un bloque `omp single`:

```
#pragma omp parallel default(none) \
    shared(a,b,x,x_new,n,norm2,...)
{
    for (iter=0; iter<max_iter; iter++) {
        double local = 0.0;
        #pragma omp for schedule(static)
        for (int i=0; i<n; i++) { ... }
        #pragma omp atomic
        norm2 += local;
        #pragma omp barrier
        #pragma omp single
        { memcpy(x,x_new,...); ... }
    }
}
```

Se estudian también `schedule(dynamic,16)`, `schedule(guided)` y la alternativa `#pragma omp critical` para la reducción.

III. METODOLOGÍA EXPERIMENTAL

Los experimentos se ejecutan en FinisTerae III mediante trabajos SLURM en la partición `short`. El tiempo se mide en **ciclos de reloj** con `get_counter()` basada en el TSC, englobando sólo el cómputo iterativo. Para cada configuración se toman **10 medidas independientes** y se selecciona la **mediana** para reducir perturbaciones del SO.

Parámetros fijos: $\text{tol}=10^{-5}$, $\text{max_iter}=15000$, `srand(42)`, dominancia diagonal garantizada sumando la suma de fila al elemento diagonal. Niveles de optimización: `-O0` y `-O3`; v3 añade `-mavx2 -mfma`; v4 usa `-fopenmp`. Hilos evaluados: {1, 2, 4, 8, 16, 32}.

La Tabla 1 resume las medianas obtenidas.

IV. RESULTADOS Y ANÁLISIS

A. Efecto de las optimizaciones de caché

La Figura 1 muestra el *speedup* de v2 respecto a v1 con ambas versiones en `-O0`. El cociente es prácticamente 1.0: las optimizaciones manuales no producen ganancia apreciable sin `-O3`, ya que el compilador sin optimizar

Cuadro 1: Medianas de ciclos ($\times 10^9$) por versión y tamaño

Versión	$n=1250$	$n=2000$	$n=3200$
v1 -O0	57.5	221.8	901.2
v1 -O3	26.4	105.9	425.0
v2 -O0	58.4	222.3	895.2
v2 -O3	26.5	99.8	400.9
v3 -O0	42.1	162.0	661.9
v3 -O3	14.8	65.8	272.9
v4 -O3 (1T)	24.4	96.6	395.9
v4 -O3 (32T)	2.4	4.8	18.7

genera código no vectorizado donde las reorganizaciones de bucle apenas alteran la localidad efectiva.

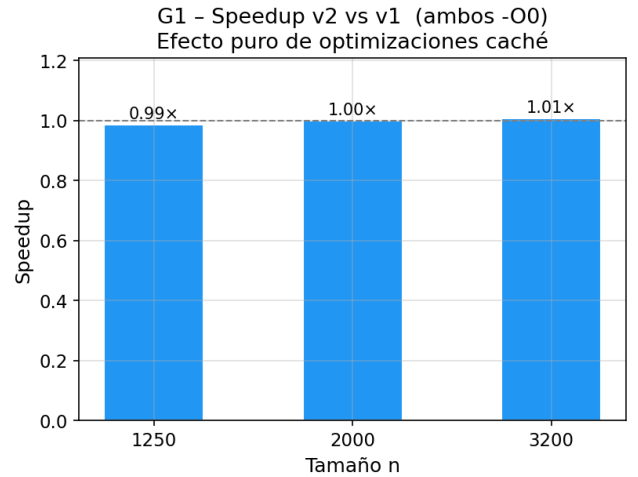


Figura 1: *Speedup* $v2/v1$ con `-O0`. Las optimizaciones de caché aportan $\approx 1\times$; el compilador las compensa sin `-O3`.

B. Todas las versiones vs. v1-O3

La Figura 2 compara v2, v3 y v4 (1 hilo) frente a v1-O3. Las versiones sin `-O3` quedan por debajo de la referencia (*speedup* < 1), confirmando que el compilador compensa muchas optimizaciones manuales. Con `-O3`, v3 destaca con *speedups* de $1.76\times$ ($n=1250$) a $1.55\times$ ($n=3200$) gracias a la vectorización AVX256. v2-O3 y v4-O3 superan ligeramente la referencia.

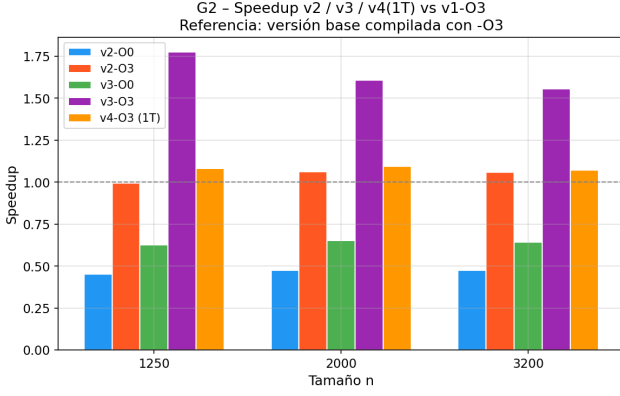


Figura 2: Speedup de v2, v3 y v4(1T) respecto a v1-03. v3-03 es la versión más rápida en todos los tamaños.

C. v3 y v4 vs. v2-03

La Figura 3 muestra el speedup sobre la mejor versión secuencial (v2-03). v3 logra $1.79\times$ para $n=1250$ y $1.47\times$ para $n=3200$; el decrecimiento con n indica que el overhead de prólogo/epílogo escalar pierde peso relativo pero la eficiencia de los registros AVX también disminuye. v4 con 1 hilo apenas supera a v2 ($1.01\text{--}1.09\times$), reflejando el overhead de barrera y single con un único hilo.

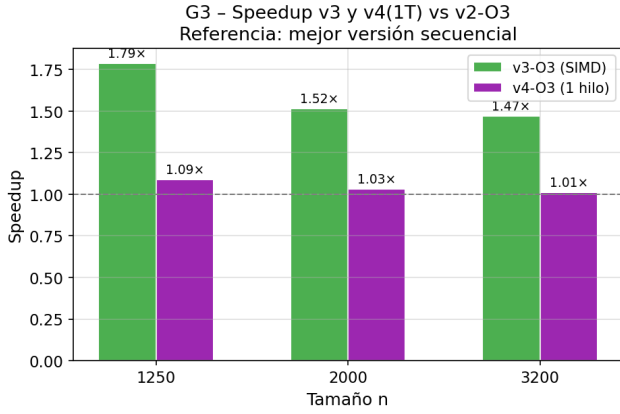


Figura 3: Speedup de v3-03 y v4-03(1T) respecto a v2-03. SIMD aporta entre $1.47\times$ y $1.79\times$.

D. Escalabilidad de v4 con el número de hilos

La Figura 4 es la gráfica **obligatoria** del enunciado. Se observa:

- Para $n=2000$ y $n=3200$, el speedup sigue la línea ideal hasta 8 hilos y alcanza $20\times$ y $21\times$ a 32 hilos, evidenciando buena escalabilidad.
- Para $n=1250$ el speedup cae a $10\times$ a 32 hilos: cada hilo procesa solo ≈ 39 filas y el overhead de sincronización supera la ganancia.
- La eficiencia paralela es $> 90\%$ hasta 8 hilos para todos los tamaños.

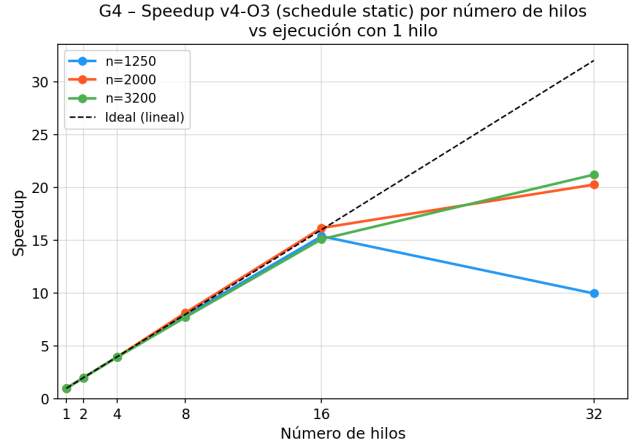


Figura 4: Speedup de v4-03 (*schedule(static)*) frente a 1 hilo para cada n . Línea discontinua: ideal lineal.

E. Comparación de schedulings

La Figura 5 compara los tres modos de reparto. Para $n=1250$ a 32 hilos, *dynamic*(16) colapsa ($\approx 3\times$) mientras *guided* alcanza $23\times$, explotando el desequilibrio natural de carga (las primeras filas tienen muchos menos elementos que las últimas). Para $n=3200$, *static* ofrece $21\times$ minimizando el overhead de scheduling cuando el desequilibrio relativo es menor.

F. Reducción atomic vs. critical

La Figura 6 muestra que *atomic* y *critical* son equivalentes en la mayoría de configuraciones: la reducción de *norm2* se ejecuta una sola vez por iteración y hilo, por lo que la contención es despreciable. La diferencia más notable aparece para $n=3200$ a 32 hilos ($25\times$ *critical* frente a $21\times$ *atomic*), apuntando a un efecto de coherencia de caché favorable a la exclusión mutua.

V. CONCLUSIONES

Se han implementado y evaluado cuatro versiones del método de Jacobi:

1. Las **optimizaciones de caché** (v2) no aportan mejora apreciable sin `-O3`: el compilador las aplica automáticamente.
2. La **vectorización AVX256+FMA** (v3) proporciona el mayor beneficio individual: entre $1.47\times$ y $1.79\times$ sobre v2-03. No alcanza el teórico $4\times$ por el overhead de prólogo/epílogo y reducción horizontal.
3. La **paralelización OpenMP** (v4) escala casi linealmente hasta 16 hilos. A 32 hilos, $n=1250$ sufre degradación por sincronización excesiva; $n=3200$ alcanza $21\times$ con *schedule(static)*.
4. *guided* es la política óptima para problemas pequeños; *static* lo es para n grande.
5. *atomic* y *critical* son prácticamente equivalentes dado el bajo nivel de contención.

Como trabajo futuro se propone combinar v3 y v4

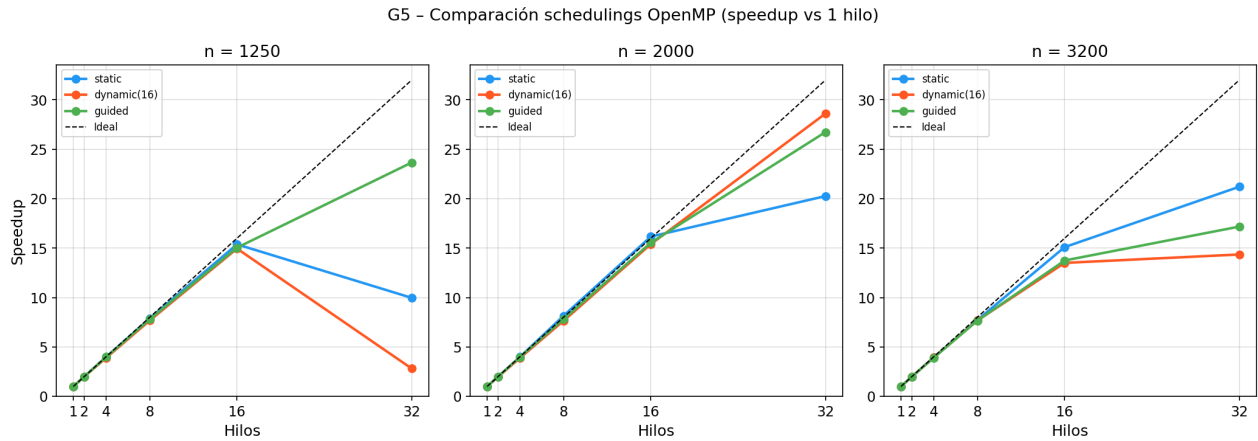


Figura 5: Comparación de schedulings OpenMP para $n \in \{1250, 2000, 3200\}$. *guided* destaca en problemas pequeños; *static* es óptimo para $n=3200$.

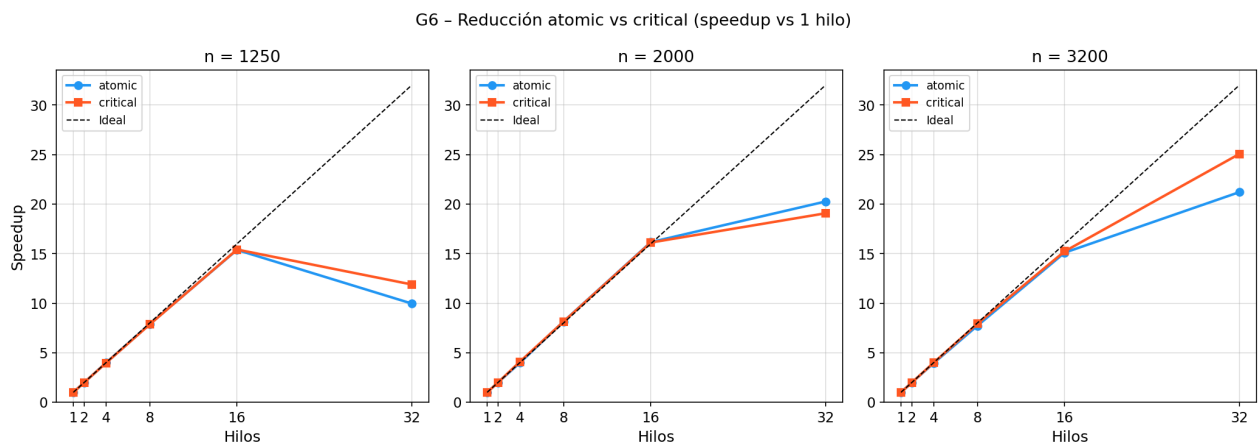


Figura 6: Comparación de reducción *atomic* vs. *critical* en v4. Diferencias pequeñas salvo a 32 hilos con $n=3200$.

(SIMD+OpenMP) y estudiar el impacto del *thread binding* (OMP_PROC_BIND) sobre la coherencia de caché en entornos NUMA.

REFERENCIAS

- [1] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, Philadelphia, 2003.
- [2] Intel Corporation, *Intel Intrinsics Guide*, <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/>, [online] última visita marzo 2026.
- [3] OpenMP Architecture Review Board, *OpenMP API v3.0 Summary Spec*, <https://www.openmp.org/wp-content/uploads/OpenMP3.0-SummarySpec.pdf>, noviembre 2008.
- [4] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design*, 5th ed. Morgan Kaufmann, 2014.